Université de Caen

UFR des Sciences - Licence Informatique 2

Projet d'Interface Graphique et Design Pattern

Jeu du Taquin

Rapport de Projet

Mise en œuvre du pattern MVC et de l'interface graphique Swing

Étudiants :
Matys ALLANET
Baptiste LE ROSSIGNOL
Minh NGUYEN

Enseignants référents :
M. MATHET
M. RANAIVOSON

Semestre 2

Table des matières

Répartition du travail **2**

1 Introduction **3**

1.1 Présentation du sujet 3

1.2 Enjeux du développement 3

2 Architecture et Design Patterns **4**

2.1 Le choix du MVC 4

2.2 L'Écouteur 4

3 Conception du Modèle **5**

3.1 Représentation des données 5

3.2 Algorithmique de mouvement 5

3.3 Garantie de solubilité 5

4 Interface Graphique et Interactions **6**

4.1 La Vue : GrilleView 6

4.2 Le Contrôleur 6

5 Conclusion **7**

Répartition du travail

Afin de mener à bien ce projet tout en respectant les principes du model MVC, nous avons divisé les tâches selon les trois couches logicielles. Cette organisation a permis un développement parallèle et indépendant.

Étudiant	**Tâches réalisées**
Baptiste	**Le Modèle (Logic)** : Implémentation de la classe `Taquin` et `Piece`. Développement des algorithmes de mélange (`mix`) et de vérification de victoire (`isResolved`).
Matys	**La Vue (UI)** : Création des composants Swing (`GrilleView`, `CoupsJouesView`). Mise en place de l'écouteur (interface `EcouteurModele`). Gestion du rendu graphique (survol, couleurs).
Minh	**Le Contrôleur (Interaction)** : Développement de `PieceController`. Gestion des événements clavier et souris. Coordination du point d'entrée (`Main`) et aide pour la partie logique.

Table 1 – Tableau de répartition des responsabilités au sein du groupe.

Outils de collaboration

- **GitHub** : Utilisé pour la gestion de versions et la fusion des différentes couches du projet.
- **LaTeX** : Utilisé pour la rédaction de ce rapport technique.
- **IntelliJ IDEA** : Environnement de développement commun.
- **Maven** : Outil de gestion de package.

1. Introduction

1.1 Présentation du sujet

Le projet consiste en la réalisation d'un jeu de Taquin. Ce jeu se compose d'une grille de $N \times N$ cases contenant des pièces numérotées et une case vide. L'objectif est de réorganiser les pièces par glissements successifs vers la case vide jusqu'à obtenir l'ordre croissant.

1.2 Enjeux du développement

Ce projet avait pour objectif de nous soumettre à certaine difficulté de conception logicielle :

- **Gestion de l'état** : Représenter fidèlement la grille et assurer l'intégrité des données lors des mouvements.
- **Résolution** : Garantir que le mélange initial permet toujours d'atteindre la solution.
- **Interface Utilisateur** : Proposer une expérience fluide via la bibliothèque Swing.

2. Architecture et Design Patterns

2.1 Le choix du MVC

Pour répondre aux exigences de propreté du code, nous avons adopté l'architecture Modèle-Vue-Contrôleur. Cela permet de modifier l'interface (Vue) sans jamais toucher à la logique métier (Modèle).

2.2 L'Écouteur

Nous avons implémenté un écouteur pour synchroniser le modèle et la vue. L'interface `EcouteurModele` agit comme un contrat : toute classe souhaitant être informée d'un changement dans le Taquin doit implémenter la méthode `modelMisAJour`.

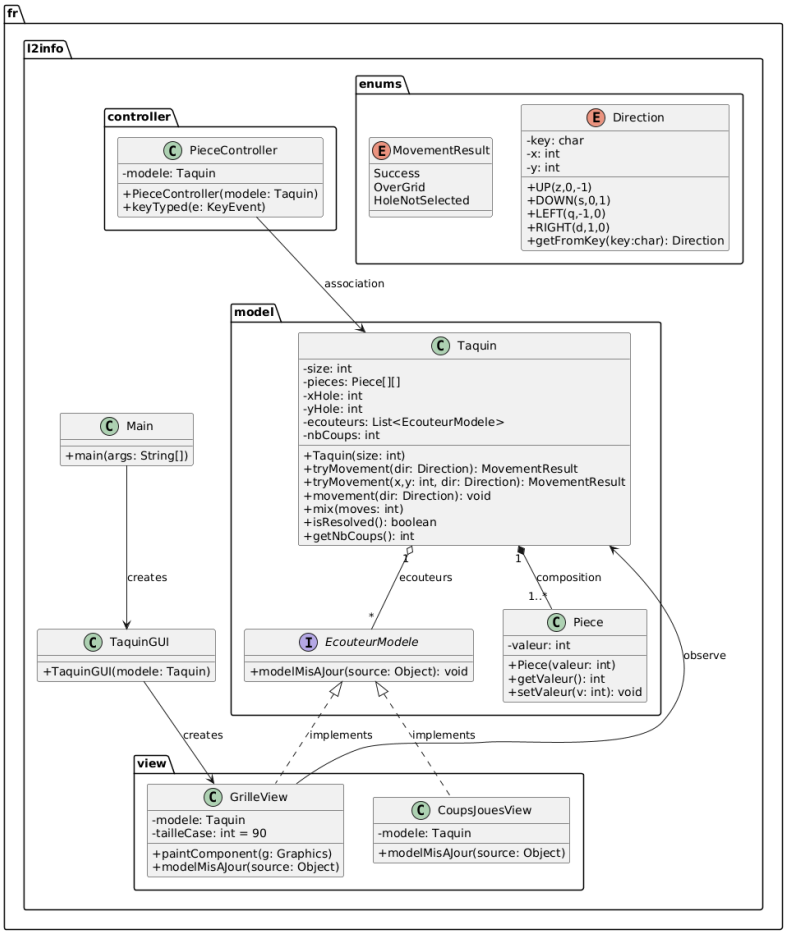

Figure 2.1 – Diagramme de classes UML illustrant la relation entre le Taquin et ses Écouteurs.

3. Conception du Modèle

3.1 Représentation des données

La grille est stockée dans une matrice d'objets `Piece`. Chaque pièce encapsule sa valeur. La valeur -1 est utilisée pour représenter la case vide (le trou).

3.2 Algorithmique de mouvement

La méthode `tryMovement` est le moteur du jeu. Elle n'effectue le mouvement que si deux conditions sont réunies :

1. La case cible est bien dans les limites de la grille.
2. Soit la case de départ, soit la case d'arrivée.

3.3 Garantie de solubilité

Le mélange ne se fait pas par un placement aléatoire des pièces (ce qui pourrait rendre le puzzle impossible), mais par une simulation de N mouvements réels à partir du trou.

4. Interface Graphique et Interactions

4.1 La Vue : GrilleView

La `GrilleView` utilise un `GridLayout` pour assurer une bonne symétrie des cases. Nous avons personnalisé les composants pour améliorer l'expérience utilisateur :

— **Code couleur** : Le trou est noir, les pièces déplaçables sont vertes.

— **Survol** : L'utilisation de `MouseMotionListener` permet de mettre en évidence la case sous le curseur.

4.2 Le Contrôleur

Le `PieceController` intercepte les clics sur les boutons. Il traduit un clic sur une coordonnée (x, y) en un appel à `tryMovement` dans le modèle. Le modèle effectuant la vérification, le contrôleur ne servant d'intermédiaire.

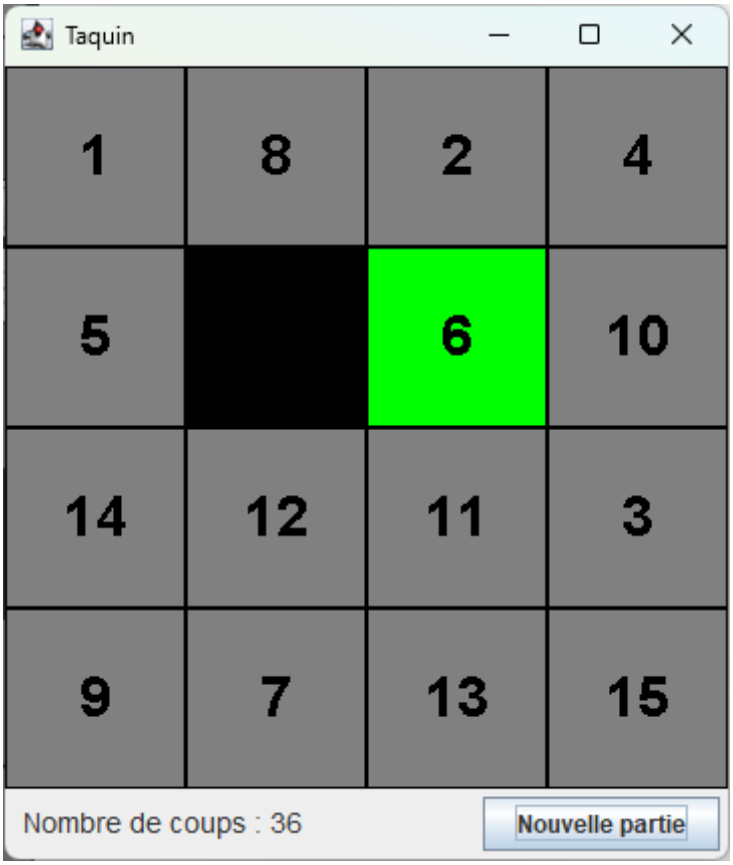

Figure 4.1 – Capture d'écran de l'interface en cours de partie.

5. Conclusion

Ce projet a permis de valider nos compétences en POO et en architecture logicielle. La mise en place du MVC a permis de nous aider à comprendre comment un vrai application fonctionne de manière pratique.

Au final, nous avons un jeu fonctionnel qui est parfaitement utilisable.